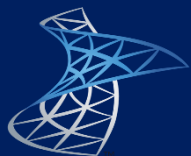


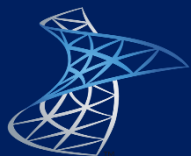
## Agenda: Particionamento de Tabelas

- Introdução ao Particionamento de Tabelas
- Arquitetura
- Benefícios
- Componentes
- Boas práticas
- Como criar
- Metadata
- **Hands On:** Particionando Tabelas

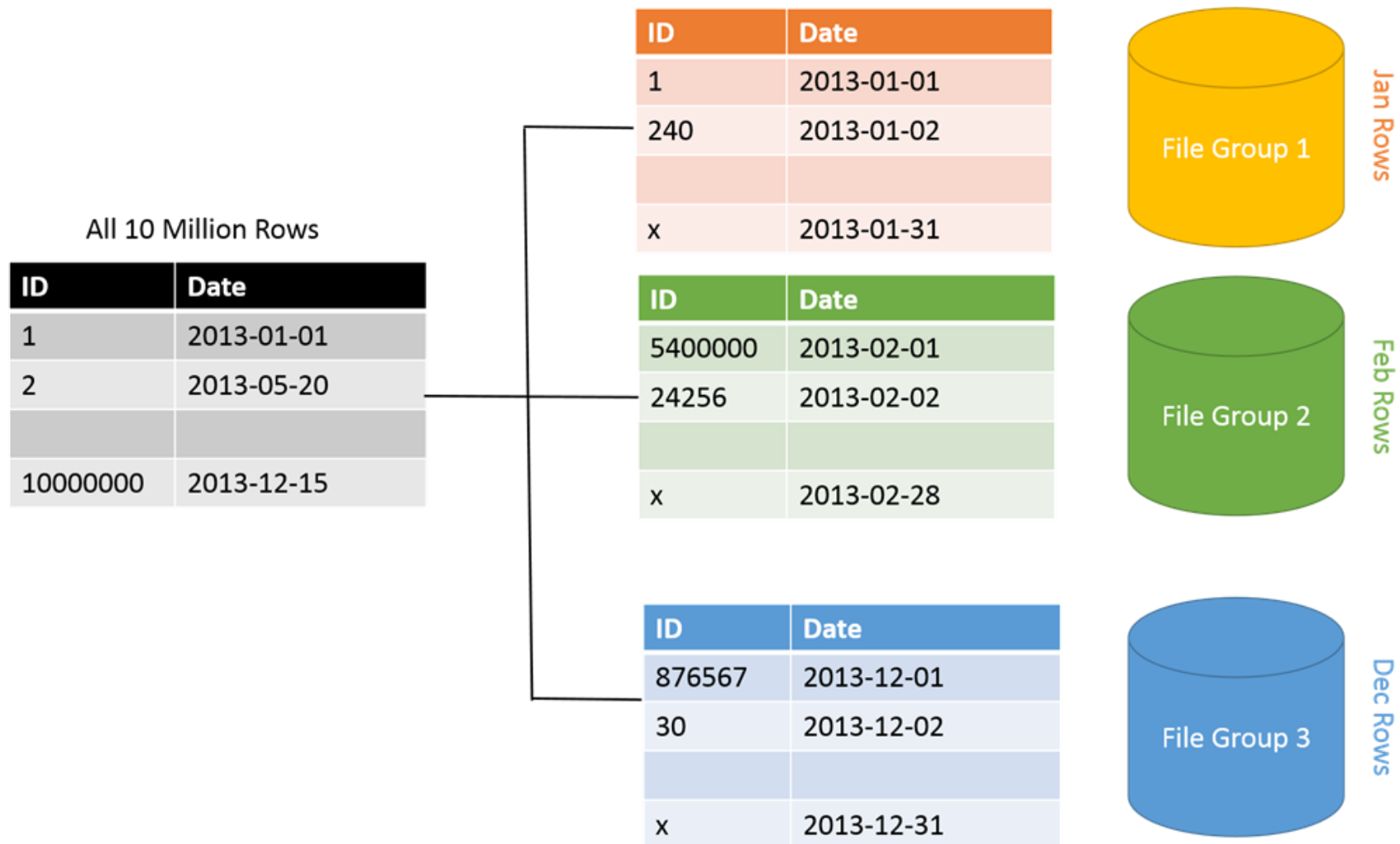


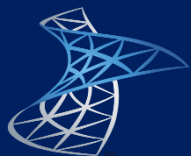
- **Introdução ao Particionamento de Tabelas**

- ✓ Disponível apenas na edição Enterprise até SQL Server 2014.
- ✓ Disponível em todas as edições a partir do SQL Server 2016 SP1, porém paralelismo apenas na edição Enterprise.
- ✓ Subdivisão de uma tabela ou índice em partes (partições), segundo critério definido pelo administrador.
- ✓ Pode armazenar cada partição em Filegroups diferentes, localizando os arquivos em discos diferentes.



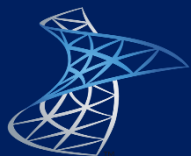
- Arquitetura





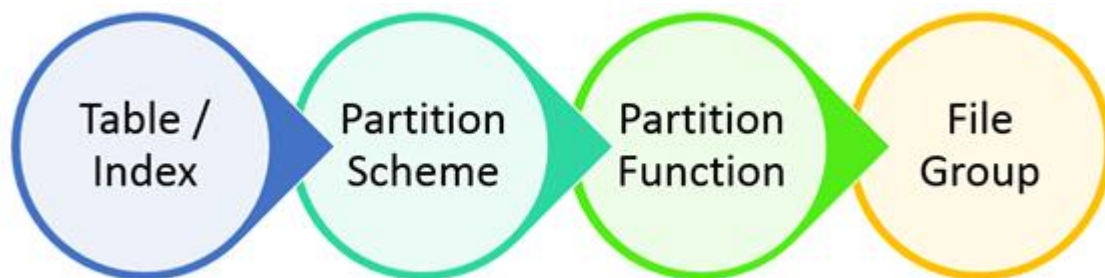
- **Benefícios**

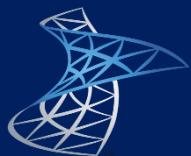
- ✓ Carga de dados mais rápida, devido a redução dos Locks.
- ✓ Operações de manutenção mais rápidas, pois podem ser executadas por partição, como: Rebuild e Reorganize de índices.
- ✓ Melhora no desempenho de consultas (será visto em detalhes em seguida).



- **Componentes**

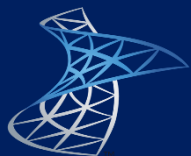
- ✓ **Partition function:** critério de divisão em partições.
- ✓ **Partition scheme:** define os filegroups que podem ser utilizados pelas partições.
- ✓ **Partitioning column:** coluna da tabela utilizada como critério de particionamento.
- ✓ **Aligned index:** índice particionado que utiliza mesmo critério de particionamento.
- ✓ **Nonaligned index:** índice particionado que utiliza critério de particionamento diferente da tabela.





- **Boas práticas**

- ✓ Mínimo de 16GB de RAM para tabelas com muitas partições.
- ✓ Limite de 15.000 partições **Não** deve ser utilizado, máximo indicado 1.000.
- ✓ Para melhor desempenho de operações em paralelo, definir o número de partições inferior a quantidade de Cores, até o máximo de 64.



## • Criação: PARTITION FUNCTION

- ✓ Define quantidade de partições e critério de divisão das linhas.

### RANGE RIGHT

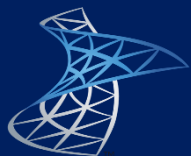
```
CREATE PARTITION FUNCTION pf_Particao (int)
AS RANGE RIGHT
FOR VALUES (2006, 2007, 2008)
```

| Partições | RANGE RIGHT        | Dados          |
|-----------|--------------------|----------------|
| 1         | < 2006             | Até 2005       |
| 2         | >= 2006 and < 2007 | 2006           |
| 3         | >= 2007 and < 2008 | 2007           |
| 4         | >= 2008            | 2008 em diante |

### RANGE LEFT

```
CREATE PARTITION FUNCTION pf_Particao (int)
AS RANGE LEFT
FOR VALUES (2006, 2007, 2008)
```

| Partições | RANGE LEFT         | Dados          |
|-----------|--------------------|----------------|
| 1         | <= 2006            | Até 2006       |
| 2         | > 2006 and <= 2007 | 2007           |
| 3         | > 2007 and <= 2008 | 2008           |
| 4         | > 2008             | 2009 em diante |



## • Criação: PARTITION SCHEME

- ✓ Define em que FileGroup cada partição será criada.
- ✓ Possui duas opções.

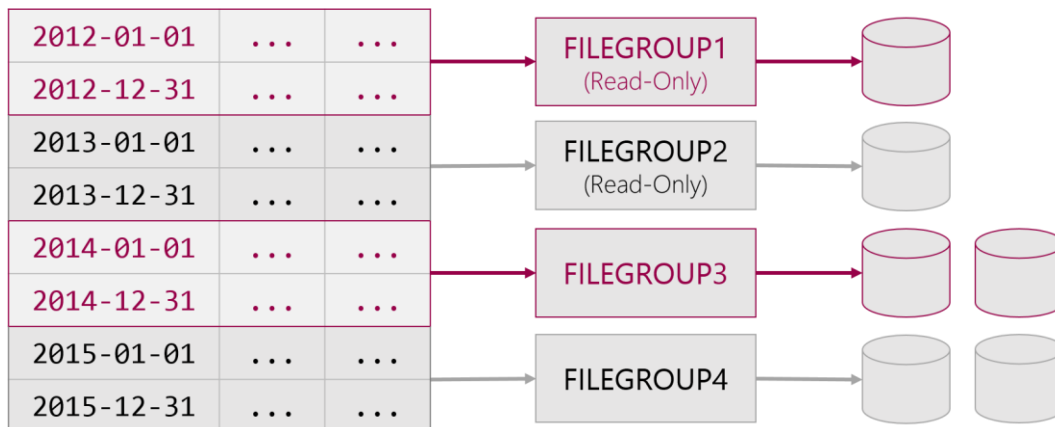
### Partições no mesmo FileGroup

```
CREATE PARTITION SCHEME ps_Particao  
AS PARTITION pf_Particao  
ALL TO ([PRIMARY])
```

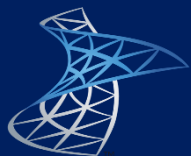
### Cada partição em um FileGroup

```
CREATE PARTITION SCHEME ps_Particao  
AS PARTITION pf_Particao  
TO (Fg1,Fg2,Fg3)
```

- ✓ Partições com histórico basta tornar FILEGROUP Read-Only.







- **Criação: CREATE TABLE**

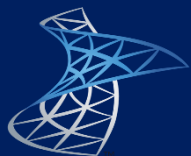
✓ Para criar tabela particionada basta associar a PARTITION SCHEME.

```
CREATE TABLE Vendas (  
  Ano int NOT NULL,  
  DataVenda_ID int NOT NULL,  
  Cliente_ID int NOT NULL,  
  ValorVenda decimal(10,2) NULL)  
ON ps_Particao(Ano)
```

✓ SIS.PARTITIONS

```
SELECT object_name(object_id), partition_id,  
partition_number, rows, data_compression_desc  
FROM sys.Partitions  
WHERE [object_id] = OBJECT_ID('dbo.Vendas')
```

| Results  |                  |                    |                  |      |                       |
|----------|------------------|--------------------|------------------|------|-----------------------|
| Messages |                  |                    |                  |      |                       |
|          | (No column name) | partition_id       | partition_number | rows | data_compression_desc |
| 1        | Vendas           | 648518351915253760 | 1                | 2    | NONE                  |
| 2        | Vendas           | 720575945994141696 | 2                | 2    | NONE                  |
| 3        | Vendas           | 792633540073029632 | 3                | 2    | NONE                  |
| 4        | Vendas           | 864691134151917568 | 4                | 2    | NONE                  |



- **Metadata: \$Partition.<partition scheme>**

✓ Função que identifica em que partição cada linha da tabela está.

```
SELECT *, $Partition.pf_Particao(Ano) Particao
FROM dbo.Vendas
```

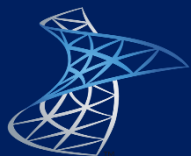
Results Messages

|   | Ano  | DataVenda_ID | Cliente_ID | ValorVenda | Particao |
|---|------|--------------|------------|------------|----------|
| 1 | 2005 | 20050110     | 1          | 100300.00  | 1        |
| 2 | 2005 | 20050210     | 2          | 2500.00    | 1        |
| 3 | 2006 | 20060110     | 2          | 40500.00   | 2        |
| 4 | 2006 | 20060210     | 3          | 65200.00   | 2        |
| 5 | 2007 | 20070110     | 3          | 30000.00   | 3        |
| 6 | 2007 | 20070210     | 4          | 115000.00  | 3        |
| 7 | 2008 | 20080110     | 4          | 230100.00  | 4        |
| 8 | 2008 | 20080210     | 5          | 15000.00   | 4        |

```
SELECT $Partition.pf_Particao(Ano) as Particao,
max(Ano) as DateKey_MAX, min(Ano) as DateKey_MIN
FROM dbo.Vendas
GROUP BY $Partition.pf_Particao(Ano)
```

Results Messages

|   | Particao | DateKey_MAX | DateKey_MIN |
|---|----------|-------------|-------------|
| 1 | 1        | 2005        | 2005        |
| 2 | 2        | 2006        | 2006        |
| 3 | 3        | 2007        | 2007        |
| 4 | 4        | 2008        | 2008        |



- **Manutenção nas Partições**

- ✓ SQL Server disponibiliza algumas operações de manutenção em tabelas particionadas para facilitar a administração.
- ✓ **MERGE**: junta duas partições em uma.
- ✓ **SPLIT**: divide uma partição em duas.
- ✓ **SWITCH**: converte uma partição em tabela não particionada (ou o inverso), apenas com alteração de metadata.



- **Hands On:** Particionando Tabelas

